

Autoencoders (AEs) and Variational Autoencoders (VAEs)

Unsupervised Representation Learning Models

Variational Autoencoders (VAEs): Decoder

Once $\mu(x)$ and $\sigma^2(x)$ of $q(z | x)$ are learned by the encoder then

$$z \sim q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

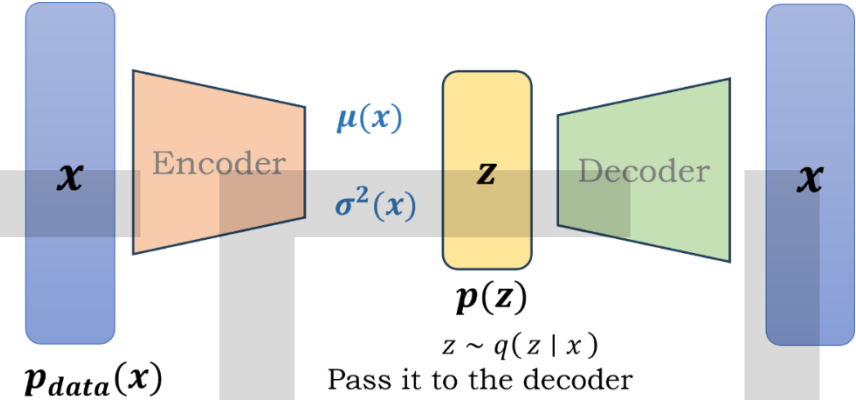
→ Sampling a latent point from this gaussian distribution

The latent variable z is randomly drawn. Given z , the decoder models the **decoder distribution**: $p(x | z)$

The decoder is a neural networks that takes z as input and outputs the **parameters of the distribution** $p(x | z) = \mathcal{N}(\mu(z), \sigma^2(z))$

Output distribution parameters depends on type of data:

Continuous data (images with pixel values 0-255 or 0-1)	Binary data (black/white images)
$p(x z) = \mathcal{N}(\mu(z), \sigma^2(z))$	$p(x z) = \text{Bernoulli}(\pi(z))$
Decoder outputs the parameters of a gaussian: $\mu(z)$ and σ^2 is often set to 1 or a small	Decoder outputs: $\pi(z)$ (probabilities)
Loss function: MSE	Loss function: BCE
The decoder predicts the "most likely" pixel values	The decoder predicts the probability that each pixel is "on"



During TRAINING:

Input → Encoder → $\mu(x), \sigma^2(x)$

Sample $z \sim q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$

Decoder takes z → outputs distribution parameters of $p(x | z)$

Compute loss directly between input and decoder mean

The decoder outputs the parameters of the distribution $p(x | z)$. The final reconstructed data point is obtained by sampling from this distribution, although in practice the mean is often used as the reconstruction.

The goal: Reconstructed data x should be as close as possible to the original input x

This closeness is measured by the **log-likelihood term**:

$$\mathbb{E}_{q(z | x)}[\log p(x | z)]$$

Variational Autoencoders (VAEs): Loss Function

The first term in the VAE Loss is log-likelihood. Always we expect the model to assign high probability to the observed data.

$$i.e. \max \mathbb{E}_{q(z|x)} [\log p(x|z)]$$

Neural networks are trained to minimize the loss not maximize. So, we convert max into minimum.

$$\max(f) \Leftrightarrow \min(-f)$$

$$L(x) = -\underbrace{\mathbb{E}_{q(z|x)} [\log p(x|z)]}_{\text{Reconstruction / Data consistency}} + \underbrace{KL(q(z|x) \parallel p(z))}_{\text{Regularization}}$$

Reconstruction / Data consistency

Mean square error

Loss function depends on a random variable z .
 z is sampled from a distribution:

$$z \sim q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

- Sampling is a random operation ✓
- Randomness breaks gradient flow, making backpropagation impossible ✓

In VAE, we only know the encoder distribution $q(z|x)$ and a simple distribution $p(z)$ as a *reference*. The KL term forces the encoder's latent space to match this reference distribution.

Regularization

$$L = L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0,1))$$

$$L_{KL} = \frac{1}{2} [\mu^2 + \sigma^2 - \log \sigma^2 - 1]$$

For the 2 gaussian distribution, the KL divergence will take this form

So, the question is: **How do we compute gradients when part of the network involves sampling?**

This motivates the Reparameterization Trick

Variational Autoencoders (VAEs): Example

Encoder Forward Pass

Input to Hidden Layer (3 neurons)

Input: $x = [5.1, 3.5, 1.4, 0.2]$

Weights $W1$ (4×3): $\begin{bmatrix} 0.1, -0.2, 0.3 \\ 0.4, 0.5, -0.6 \\ -0.7, 0.8, 0.9 \\ 0.2, -0.1, 0.4 \end{bmatrix}$

Let Bias ($b1$) = 0.1
activation function

$$h_1 = (5.1 \times 0.1) + (3.5 \times 0.4) + (1.4 \times -0.7) + (0.2 \times 0.2) + 0.1 = 1.07$$

$$h_2 = (5.1 \times -0.2) + (3.5 \times 0.5) + (1.4 \times 0.8) + (0.2 \times -0.1) + 0.1 = 1.93$$

$$h_3 = (5.1 \times 0.3) + (3.5 \times -0.6) + (1.4 \times 0.9) + (0.2 \times 0.4) + 0.1 = 0.87$$

$$h_1 = 1.07, h_2 = 1.93, h_3 = 0.87$$

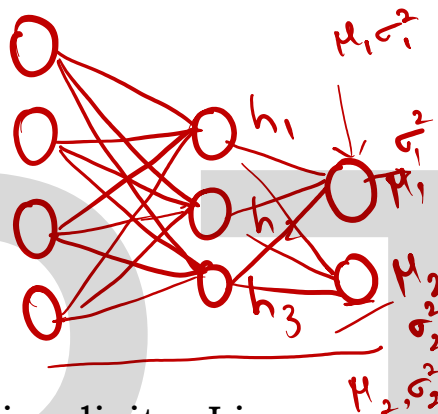
Latent dimension: 2

$$\mu = hw_{\mu} + b \quad w_{\mu}: [[0.5, 0.2], [-0.3, 0.4], [0.1, -0.5]], b=0.0$$

$$\mu_1 = 1.07 \times 0.5 + 1.93 \times (-0.3) + 0.87 \times 0.1 + 0.0 = 0.043$$

$$\mu_2 = 1.07 \times 0.2 + 1.93 \times 0.4 + 0.87 \times (-0.5) + 0.0 = 0.551$$

Result: $\mu = [0.043, 0.551]$



$$w_{\sigma}: [[-0.2, 0.3], [0.4, -0.1], [0.2, 0.2]], b=0.0$$

$$\log(\sigma_1^2) = 1.07 \times (-0.2) + 1.93 \times 0.4 + 0.87 \times 0.2 + 0.0 = 0.732$$

$$\log(\sigma_2^2) = 1.07 \times 0.3 + 1.93 \times (-0.1) + 0.87 \times 0.2 + 0.0 = 0.302$$

Result: $\log(\sigma^2) = [0.732, 0.302]$

Convert to Variance and Standard Deviation

$$\sigma^2 = e^{\log(\sigma^2)} \quad \sigma = \sqrt{\sigma^2}$$

$$\sigma_1 = 1.442, \sigma_2 = 1.163$$

$$z \sim q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

$$z_1 = 0.041 \quad z_2 = 0.489$$

since $z \sim q(z | x)$ and $q(z | x)$ is a **distribution**, infinitely many latent samples are possible

Sample 1: $z^{(1)} = [0.041, 0.489] = [z_1, z_2]$

Sample 2: $z^{(2)} = [0.042, 0.549]$

Sample 3: $z^{(3)} = [0.9, 0.1]$

All these z 's come from the same distribution $q(z | x)$

	μ_1, σ_1^2	μ_2, σ_2^2
z_1	0.041	0.489
z_2		
z_3		
$\vdots$		
z^n		

Variational Autoencoders (VAEs): Example

Decoder Forward Pass

Sampled latent vector:

$$z = [z_1, z_2] = [0.041, 0.489]$$

Assume a symmetric decoder architecture:

1. Input: z (2 dimensions)
2. Hidden layer: 3 neurons
3. Output: 4 dimensions (reconstruction)

Hidden layer computation:

$$h = zW_2 + b_2$$

$$W_2 (2 \times 3) = [[0.6, -0.4, 0.2], [0.3, 0.1, -0.5]], b_2 = 0.1$$

$$\begin{aligned} h_1 &= (0.041 * 0.6) + (0.489 * 0.3) + 0.1 \\ &= 0.0246 + 0.1467 + 0.1 = 0.2713 \end{aligned}$$

$$\begin{aligned} h_2 &= (0.041 * (-0.4)) + (0.489 * 0.1) + 0.1 \\ &= -0.0164 + 0.0489 + 0.1 = 0.1325 \end{aligned}$$

$$\begin{aligned} h_3 &= (0.041 * 0.2) + (0.489 * (-0.5)) + 0.1 \\ &= 0.0082 - 0.2445 + 0.1 = -0.1363 \end{aligned}$$

$$\text{So } h = [0.2713, 0.1325, -0.1363]$$

$$W_3 (3 \times 4) = [[0.2, -0.1, 0.3, 0.4], [0.1, 0.2, -0.2, 0.1], [-0.3, 0.4, 0.1, -0.2]], b_3 = 0.0$$

Decoder output:

$$x = \mu_x = hW_3 + b_3$$

$$\begin{aligned} x_1 &= h_1 * 0.2 + h_2 * 0.1 + h_3 * (-0.3) \\ &= 0.2713 * 0.2 + 0.1325 * 0.1 + (-0.1363) * (-0.3) \\ &= 0.05426 + 0.01325 + 0.04089 = 0.1084 \end{aligned}$$

$$\begin{aligned} x_2 &= h_1 * (-0.1) + h_2 * 0.2 + h_3 * 0.4 \\ &= 0.2713 * (-0.1) + 0.1325 * 0.2 + (-0.1363) * 0.4 \\ &= -0.02713 + 0.0265 - 0.05452 = -0.05515 \end{aligned}$$

$$\begin{aligned} x_3 &= h_1 * 0.3 + h_2 * (-0.2) + h_3 * 0.1 \\ &= 0.2713 * 0.3 + 0.1325 * (-0.2) + (-0.1363) * 0.1 \\ &= 0.08139 - 0.0265 - 0.01363 = 0.04126 \end{aligned}$$

$$\begin{aligned} x_4 &= h_1 * 0.4 + h_2 * 0.1 + h_3 * (-0.2) \\ &= 0.2713 * 0.4 + 0.1325 * 0.1 + (-0.1363) * (-0.2) \\ &= 0.10852 + 0.01325 + 0.02726 = 0.14903 \end{aligned}$$

$$\text{So reconstructed } x = [0.1084, -0.05515, 0.04126, 0.14903]$$

$$\mu_x = [0.1084, -0.05515, 0.04126, 0.14903]$$

This represents the **mean of the reconstruction distribution**:

$$p(x | z) = \mathcal{N}(\mu_x, \sigma_x^2)$$

$p(x|z)$
↓
mean

Variational Autoencoders (VAEs): Example

Final Reconstruction (Conceptual)

➤ **Theoretically**, the reconstructed data is sampled from:

$$\underline{x_{reconstructed}} \sim \underline{p(x | z)}$$

➤ **In practice**, the mean of this distribution itself is the reconstructed data

$$\underline{x_{reconstructed}} \approx \underline{\mu_x}$$

So for the numerical example:

$$x_{reconstructed} = [0.1084, -0.05515, 0.04126, 0.14903] \quad \checkmark$$

Original input:

$$\underline{x = [5.1, 3.5, 1.4, 0.2]} \quad \checkmark$$

The reconstruction is poor because the weights are randomly initialized and the model has not been trained yet. After training—when the learnable parameters are optimized—the reconstructed data will be much closer to the original input.

The decoder outputs the parameters of $p(x | z)$.

The final reconstructed data point is obtained by sampling from this distribution, although in practice the mean is often used as the reconstruction.

Variational Autoencoders (VAEs): Example

Reconstruction Loss

Since we assumed a **Gaussian decoder** and used the **decoder mean** μ_x as reconstruction, we compute:

$$\begin{aligned}\text{Reconstruction Loss} &= \frac{1}{4} \sum_{i=1}^4 (x_i - \mu_{x,i})^2 \\ &= \frac{1}{4} \left[(5.1 - 0.1084)^2 + (3.5 + 0.05515)^2 + (1.4 - 0.04126)^2 + (0.2 - 0.14903)^2 \right] \\ &= \frac{1}{4} [24.916 + 12.639 + 1.846 + 0.0026] \\ &= \frac{39.4036}{4} = 9.8509\end{aligned}$$

KL Divergence Loss

The KL divergence is:

$$\begin{aligned}L &= L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) \\ L_{KL} &= \frac{1}{2} [\mu^2 + \sigma^2 - \log \sigma^2 - 1]\end{aligned}$$

Compute:

$$\begin{aligned}\mu_1^2 &= 0.00185, \mu_2^2 = 0.3036 \\ \text{KL} &= \frac{1}{2} [(0.00185 + 2.079 - 0.732 - 1) + (0.3036 + 1.353 - 0.302 - 1)] = \frac{1}{2} [0.349 + 0.355] = \frac{1}{2} (0.704) = \underline{0.352}\end{aligned}$$

Encoder Outputs

$$\begin{aligned}\mu &= [0.043, 0.551] \\ \log \sigma^2 &= [0.732, 0.302] \\ \sigma^2 &= [2.079, 1.353]\end{aligned}$$

$z \sim q(z|x)$

Total VAE Loss

$$\begin{aligned}&= \text{Reconstruction Loss} + \text{KL Divergence} \\ &= 9.8509 + 0.352 = \underline{10.2029}\end{aligned}$$

Variational Autoencoders (VAEs): Example

Multiple latent samples for the same input

$$z^{(1)} = [0.041, 0.489] \xrightarrow{\text{Decoder}} \mu_x^{(1)} = [0.1084, -0.05515, 0.04126, 0.14903]$$

$$z^{(2)} = [0.042, 0.549] \xrightarrow{\text{Decoder}} \mu_x^{(2)} = [\dots]$$

$$z^{(3)} = [0.9, 0.1] \xrightarrow{\text{Decoder}} \mu_x^{(3)} = [\dots]$$

Each z generates a **different reconstruction distribution**

$$p(x | z)$$

Which latent variable z produced the observed data x ?

$$p(x) = \int p(x | z) \cdot p(z) dz$$

The probability of observing x is obtained by summing (integrating) the contributions from **all possible latent variables**, weighted by how likely each z is under the prior $p(z)$.

- There are infinitely many possible z 's ✓
- Each z produces a different $p(x | z)$ ✓
- The integral has no closed-form solution ✓
- Therefore, computing $p(x)$ exactly is intractable.

∴ computing $p(x)$ is very hard

What do we actually want?

We want the true posterior:

$$p(z | x) = \frac{p(x | z) p(z)}{p(x)} \quad \checkmark$$

But since $p(x)$ is intractable,

We cannot compute $p(z | x)$ directly

So we introduce a neural-network approximation:

$$q(z | x) \approx p(z | x)$$

This is what the **encoder learns**.

Next Session

➤ Reparameterization Trick ✓

$$\checkmark \underline{z = \mu + \sigma \times \varepsilon}$$

$$\checkmark \underline{z_1 = \mu_1 + \sigma_1 \times \varepsilon_1 = 0.043 + 1.442 \times 0.5 = 0.043 + 0.721 = 0.764}$$

$$\checkmark \underline{z_2 = \mu_2 + \sigma_2 \times \varepsilon_2 = 0.551 + 1.163 \times (-0.2) = 0.551 - 0.233 = 0.318}$$

Sampled $\mathbf{z}$: [0.764, 0.318]

$$z \sim q(z|x)$$

NIPTELL

Thank you